

Laboratorio

semana 3

NovaTerra



Equipo {

LUIS
STIVEN
MOYANO
VERDUGO

MANUEL
FELIPE
RINCON
MANRIQUE

}

Contenidos

01

Problema

02

RQF y RQNF

03

HU

04

Diagrama de flujo

05

Diagrama de secuencia

06

Diagrama de uso

07

Punto clave

Problema {

Una tienda maneja una lista con los códigos numéricos de sus productos. Como los productos se registran a medida que llegan, los códigos inicialmente se encuentran desordenados.

El encargado necesita desarrollar un programa que permita:

1. Registrar una cantidad n de productos.
2. Ordenar sus códigos utilizando Merge Sort.
3. Buscar un código específico utilizando Búsqueda Binaria.



}

RQF{

En este problema pudimos identificar tres funciones principales

- RQF01 Leer la cantidad del producto
- RQF02 Leerlos codigos del producto
- RQF03 Leer el codigo a buscar



}

RQNF{

RQNF 01 (Eficiencia): Al estar usando métodos de organización como el merge Sort y el de búsqueda como búsqueda binaria, el código reduce en gran parte su complejidad, por lo cual ocupa menos memoria ram

RQNF 02 (Mantenibilidad): Ya que el código es modular, es más fácil adaptarlo, modificarlo

RQNF 03 (Corrección): Ya que el código es modular, permite ver las funciones fácilmente si hay un error

RQNF 04 (Programar en Python): Programamos en Python porque el lenguaje Python es un lenguaje fácil de comprender el cual se usa más que todo en programas de datos



HU 1{

HU 01 : Leer la cantidad del producto

RQF01

Entradas

→ cantidad del producto

Salidas

→ un mensaje que dice "Dato valido"

→ Un mensaje que dice "Dato invalido, por favor asegurese de ingresar solo:

- Numeros enteros
- Numeros positivos
- No dejar el campo vacio
- No ingresar letras ni simbolos
- Introducir un solo numero
- No usar espacios, comas o cualquier otro caracter

CA

→ número positivo

CE

→ Si no es número

→ Si es un número negativo

→ Si el numero tiene un espacion o caracter adicional



}

HU 2{

HU 02: Leerlos codigos del producto

RQF02

Entradas

→ codigos del producto

Salidas

→ un mensaje que dice "Dato valido"

→ Un mensaje que dice "Dato invalido, por favor asegurese de ingresar solo:

- Numeros enteros
- Numeros positivos
- No dejar el campo vacio
- No ingresar letras ni simbolos
- Introducir un solo numero
- No usar espacios, comas o cualquier otro caracter

CA

→ número positivo

CE

→ Si no es número

→ Si es un número negativo

→ Si el numero tiene un espacion o caracter adicional



}

HU 3{

HU 03: Leer el código a buscar

RQF03

Entradas

→ código del producto

Salidas

→ Un mensaje diciendo "El número está en la lista y en qué posición"

→ Un mensaje diciendo "El número no está en la lista"

→ Un mensaje que dice "Dato inválido, por favor asegúrese de ingresar solo:

- Números enteros
- Números positivos
- No dejar el campo vacío
- No ingresar letras ni símbolos
- Introducir un solo número
- No usar espacios, comas o cualquier otro carácter

CA

→ número positivo

CE

→ Si no es número

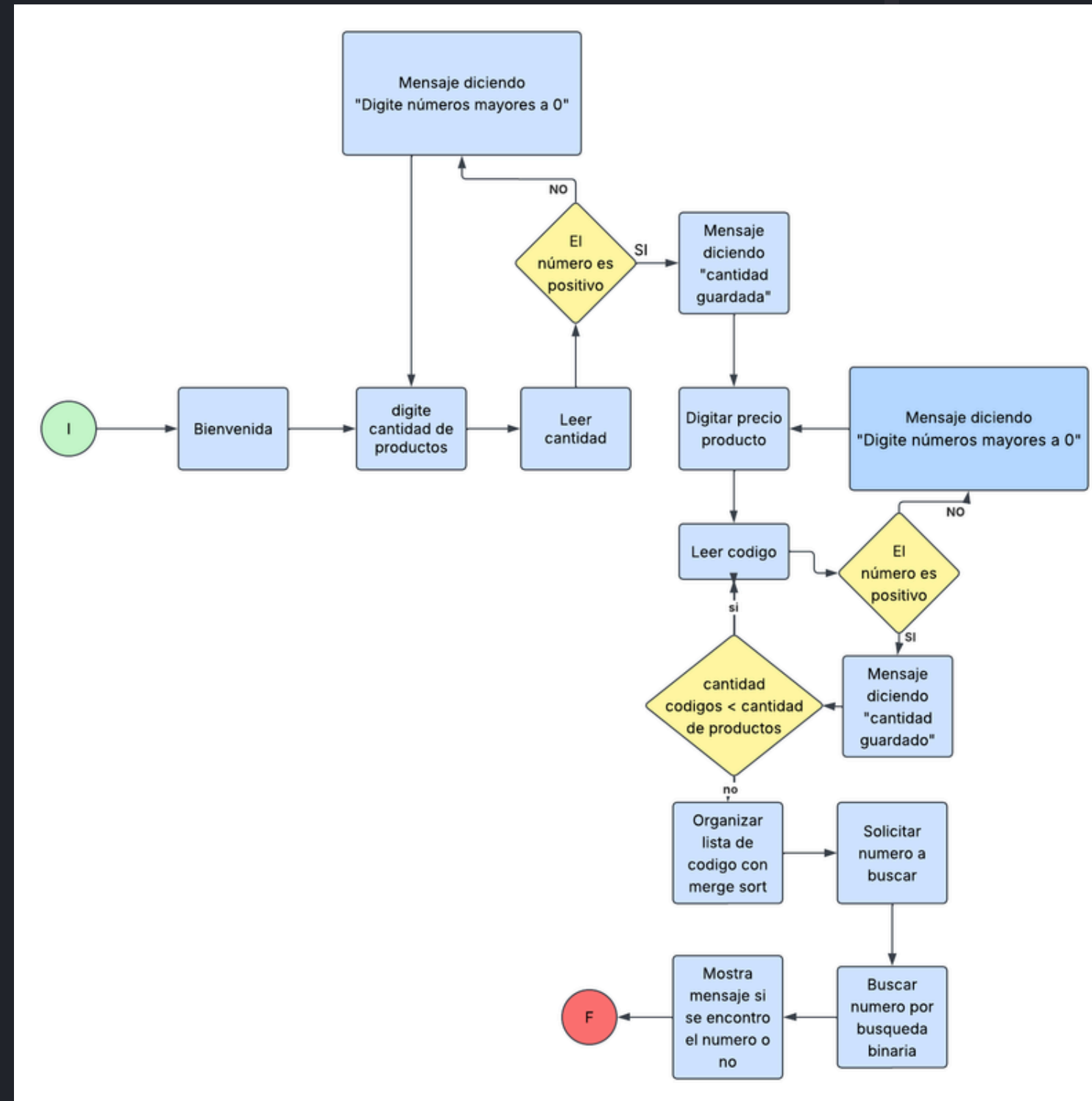
→ Si es un número negativo

→ Si el número tiene un espacio o carácter adicional



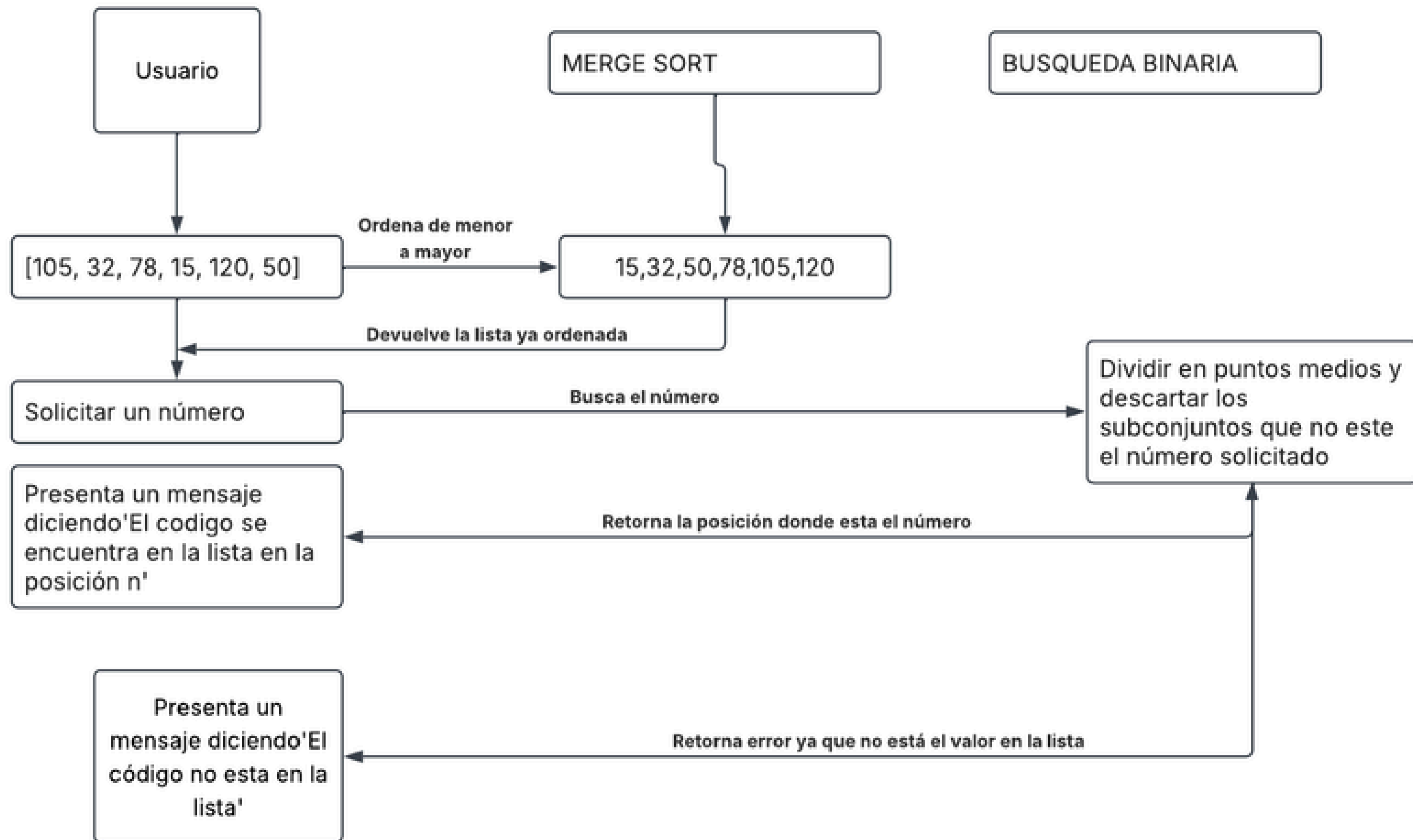
}

Diagrama de flujo{



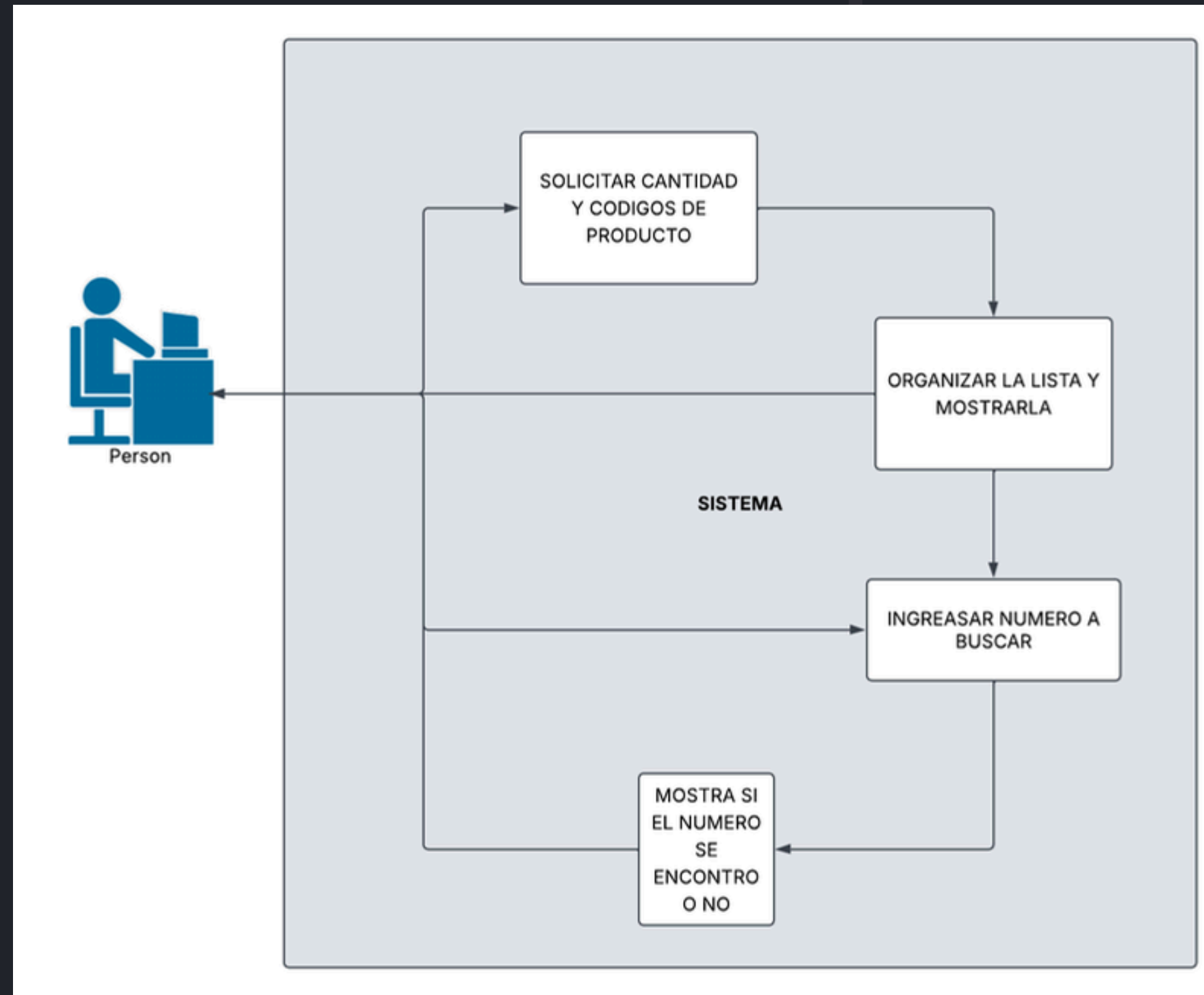
}

Diagrama de secuencia{



}

Diagrama de uso{



}

Merge Sort{

En el merge sort divide el conjunto en 2 partes iguales hasta que cada subconjunto tenga 1 solo elemento. Luego la función merge recorre todos los subconjuntos para unir los elementos de manera ordenada

Teorema Maestro

$$T[n] = 2T(n/2) + \Theta(n)$$

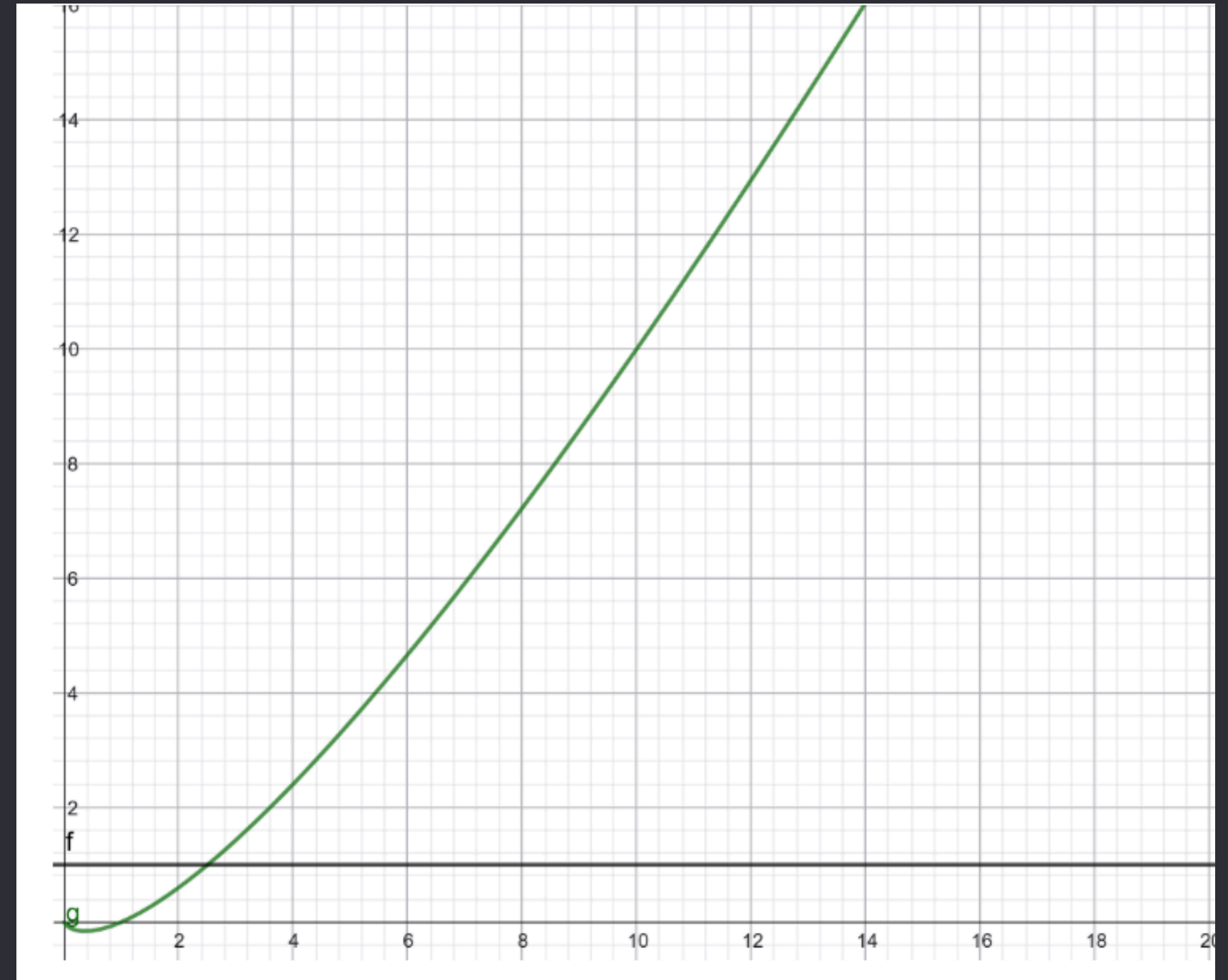
Recurrencia:

$$T[n] = \Theta(n^{\log_2(2)} * \log^{(0+1)}(n))$$

$$T[n] = \Theta(n \log(n))$$

Mejor caso = 1

Peor caso = $n \log(n)$



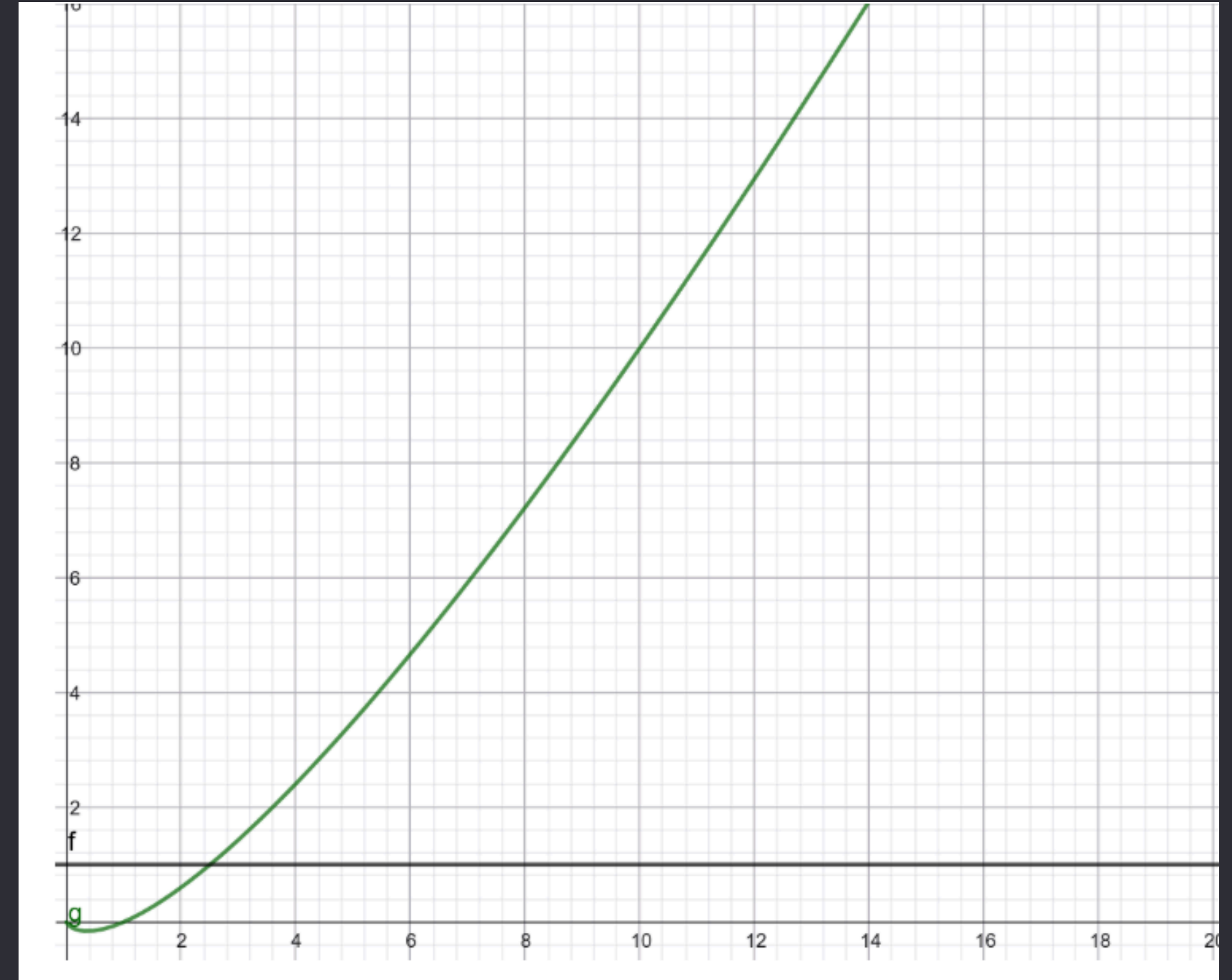
}

Busqueda Binaria

En la búsqueda binaria funciona en una lista ordenada, donde en cada paso se divide en 2 partes iguales y se descarta la parte donde no esta el elemento que queremos encontrar

Teorema maestro

- $T[n] = T(n/2) + \Theta(n)$
- **Recurrencia:**
- $T[n] = \Theta(n^{\log_2(1)} * \log^{(0+1)}(n))$
 $T[n] = \Theta(\log(n))$
- **Mejor caso** = 1
- **Peor caso** = $\log n$



<CODIGO>

Gracias